



PERTEMUAN 3

Control Statements: Part 1

Memahami dasar-dasar pengendalian alur program dalam Java — mulai dari **Assignment**, operator **++** dan **--**, hingga berbagai jenis pernyataan kontrol yang menjadi fondasi logika pemrograman.





Algoritma — Contoh Sederhana

Setiap masalah komputasi dapat diselesaikan dengan menjalankan serangkaian tindakan dalam urutan tertentu. Prosedur ini mencakup **tindakan yang harus dijalankan** dan **urutan pelaksanaannya**. Menentukan urutan tersebut disebut **kontrol program**.

Algoritma "Bangkit & Bersinar"

1. Bangun dari tempat tidur
2. Melepas piyama
3. Mandi
4. Berpakaian
5. Makan sarapan
6. Carpool untuk bekerja

Mengapa Urutan Penting?

Bayangkan jika Anda mandi setelah berpakaian — hasilnya tidak masuk akal! Dalam pemrograman, urutan eksekusi sama pentingnya. Bab ini menyelidiki bagaimana **control statements Java** mengatur urutan tersebut secara presisi.

Pseudocode

Pseudocode adalah bahasa informal yang membantu mengembangkan algoritma **tanpa harus khawatir** tentang detail ketat sintaks bahasa Java. Ini adalah alat berpikir, bukan alat eksekusi.

Untuk Berpikir

Pseudocode membantu kamu "memikirkan" sebuah program sebelum menulisnya dalam bahasa pemrograman seperti Java.

Tidak Dieksekusi

Pseudocode **tidak dijalankan** di komputer. Ia hanya berfungsi sebagai sketsa logika sebelum penulisan kode nyata dimulai.

Mudah Dikonversi

Pseudocode yang baik dirancang agar mudah dikonversi ke bagian terstruktur dari program Java dengan perubahan minimal.

 Pseudocode adalah jembatan antara ide dan kode — gunakan sebelum mulai menulis program!

Control Structure & Control Statements pada Java

Java memiliki **tiga jenis struktur kontrol** yang disebut sebagai *control statements*. Setiap jenis memiliki peran khusus dalam mengendalikan alur eksekusi program.

1

Sequence Statement

Eksekusi pernyataan secara berurutan, satu per satu dari atas ke bawah.

2

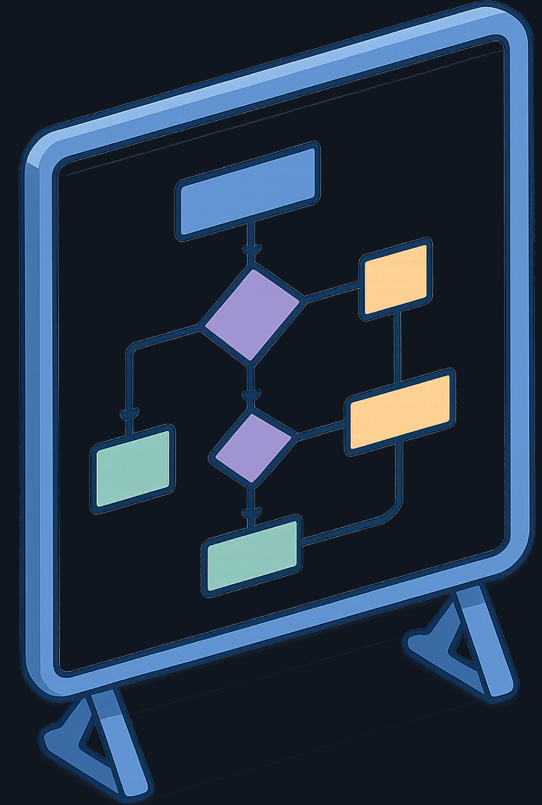
Selection Statements

Memilih jalur eksekusi berdasarkan kondisi tertentu. Terdiri dari **3 tipe**: if, if...else, dan switch.

3

Iteration Statements

Mengulang blok kode selama kondisi terpenuhi. Terdiri dari **4 tipe**: while, do...while, for, dan enhanced for.



Sequence Structure dalam Java

Struktur urutan (*sequence structure*) adalah struktur paling dasar yang **sudah tertanam langsung ke dalam Java**. Kecuali diarahkan sebaliknya oleh pernyataan kontrol lain, komputer mengeksekusi pernyataan Java **satu demi satu** sesuai urutan penulisannya.

Ini berarti baris kode yang ditulis paling atas akan dieksekusi pertama, diikuti baris berikutnya, dan seterusnya — dari atas ke bawah secara linear.

- ① Sequence adalah default behavior Java. Tanpa kontrol khusus, semua kode berjalan berurutan.



Selection Statements

Java memiliki **tiga jenis pernyataan seleksi** untuk memilih jalur eksekusi berdasarkan kondisi. Setiap jenis memiliki karakteristik dan kegunaan yang berbeda.



if — Single Selection

Melakukan suatu tindakan **jika kondisi benar**, atau melewatkannya jika kondisi salah. Disebut single-selection karena memilih atau mengabaikan satu tindakan.



if...else — Double Selection

Melakukan tindakan jika kondisi **benar**, dan tindakan berbeda jika kondisi **salah**. Disebut double-selection karena memilih antara dua alternatif.



switch — Multi Selection

Melakukan salah satu dari **banyak tindakan berbeda** bergantung pada nilai ekspresi. Disebut multi-selection karena memilih di antara banyak alternatif.

Iteration Statements

Java menyediakan **empat pernyataan iterasi** (juga disebut pernyataan pengulangan atau *loop*) yang memungkinkan program melakukan pernyataan berulang kali selama suatu kondisi — disebut **kondisi loop-continuation** — tetap bernilai benar.

1

while

Mengecek kondisi **sebelum** iterasi. Jika kondisi awal salah, blok tidak dieksekusi sama sekali.

2

do...while

Mengecek kondisi **setelah** iterasi. Blok dieksekusi **minimal satu kali** meskipun kondisi salah.

3

for

Iterasi dengan **counter** yang terstruktur. Inisialisasi, kondisi, dan update ditulis dalam satu baris.

4

Enhanced for

Versi sederhana dari for, digunakan untuk mengiterasi **array atau koleksi** secara langsung.

if — Single-Selection Statement

Program menggunakan pernyataan seleksi **if** untuk memilih di antara tindakan alternatif berdasarkan kondisi tertentu. Jika kondisi bernilai **true**, blok kode dieksekusi; jika **false**, blok dilewati.

Sintaks Dasar

```
if (kondisi) {  
    // pernyataan  
    // dieksekusi jika  
    // kondisi = true  
}
```

Contoh Nyata

```
if (nilai >= 60) {  
  
    System.out.println("Lulus!")  
    ;  
}
```

Jika nilai siswa 60 ke atas, program mencetak "Lulus!". Jika di bawah 60, tidak ada output sama sekali.



if...else — Double-Selection Statement

Pernyataan `if...else` memungkinkan Anda menentukan **dua jalur aksi**: satu tindakan saat kondisi benar, dan tindakan lain saat kondisi salah. Tidak ada kemungkinan yang "diabaikan".

Sintaks if...else

```
if (kondisi) {  
    // jika true  
} else {  
    // jika false  
}
```

Contoh Pseudocode

```
If student's grade >= 60  
    Print "Passed"  
Else  
    Print "Failed"
```

Implementasi Java

```
if (studentGrade >= 60) {  
    System.out.println("Passed");  
} else {  
    System.out.println("Failed");  
}
```

Setiap siswa pasti mendapat salah satu output: **"Passed"** atau **"Failed"** — tidak ada yang terlewat.

Nested if...else Statements

Sebuah program dapat menguji **beberapa kasus sekaligus** dengan menempatkan pernyataan `if...else` di dalam pernyataan `if...else` lainnya — inilah yang disebut **nested if...else**.

Contoh: Nilai Huruf

```
if (nilai >= 90) {  
    grade = "A";  
} else if (nilai >= 80) {  
    grade = "B";  
} else if (nilai >= 70) {  
    grade = "C";  
} else if (nilai >= 60) {  
    grade = "D";  
} else {  
    grade = "F";  
}
```

Cara Kerja Nested if...else

Java mengevaluasi kondisi **dari atas ke bawah**. Begitu satu kondisi bernilai true, blok tersebut dieksekusi dan sisa kondisi lainnya **dilewati**.

Pendekatan ini sangat efisien untuk mengelompokkan nilai ke dalam banyak kategori dengan logika yang jelas dan mudah dibaca.

📌 Nested if...else cocok saat ada banyak kategori nilai atau kondisi yang saling eksklusif.

Dangling-else Problem

Masalah **dangling-else** adalah kesalahan logika yang terjadi saat kompilator mengasosiasikan pernyataan `else` dengan `if` yang **salah**, menghasilkan perilaku program yang tidak sesuai harapan.

✗ Bermasalah (Tanpa Kurung Kurawal)

```
if (x > 5)
    if (y > 5)
        System.out.println("x & y > 5");
else // ini milik if(y>5), bukan if(x>5)!
    System.out.println("x <= 5");
```

☑ Aman (Dengan Kurung Kurawal)

```
if (x > 5) {
    if (y > 5) {
        System.out.println("x & y > 5");
    }
} else {
    System.out.println("x <= 5");
}
```

⚠ Selalu gunakan kurung kurawal { } meskipun hanya ada satu pernyataan dalam blok if atau else!

Blocks dalam Java

Sebuah **block** (blok) adalah kumpulan pernyataan yang diapit oleh kurung kurawal { }. Blok memungkinkan beberapa pernyataan dikelompokkan menjadi satu unit, khususnya di bagian `if` atau `else`.

Contoh Block pada else

```
if (nilai >= 60) {  
    System.out.println("Lulus");  
} else {  
    System.out.println("Gagal");  
    System.out.println("Ikuti remedial");  
    nilaiAkhir = 0;  
}
```

Manfaat Penggunaan Block

- Mengelompokkan beberapa pernyataan menjadi satu unit logis
- Mencegah kesalahan dangling-else
- Membuat kode lebih mudah dibaca dan dipelihara
- Memungkinkan penambahan pernyataan baru tanpa mengubah struktur

Conditional Operator (?:)

Java menyediakan **conditional operator** (?:) sebagai cara ringkas untuk menggantikan pernyataan **if...else** sederhana. Operator ini juga dikenal sebagai *ternary operator* karena membutuhkan tiga operand.

Sintaks Conditional Operator

```
kondisi ? nilaiJikaTrue : nilaiJikaFalse
```

Jika kondisi **true**, ekspresi menghasilkan nilai pertama. Jika **false**, menghasilkan nilai kedua.

Contoh Penggunaan

```
// Versi if...else panjang:  
if (studentGrade >= 60)  
    System.out.println("Passed");  
else  
    System.out.println("Failed");  
  
// Versi ringkas dengan ?:  
System.out.println(  
    studentGrade >= 60 ? "Passed" : "Failed"  
);
```

 Gunakan conditional operator hanya untuk kondisi sederhana. Untuk logika kompleks, tetap gunakan if...else agar kode mudah dibaca.

Latihan: Analisis Program

Sebelum melanjutkan ke materi berikutnya, mari kita uji pemahaman kita! Cermati kode program di bawah ini dan jelaskan **apa yang dilakukan setiap bagian kode tersebut** — mulai dari variabel yang digunakan, kondisi yang dievaluasi, hingga output yang dihasilkan.

🔍 Yang Perlu Dianalisis

- Variabel apa yang digunakan?
- Kondisi apa yang diperiksa?
- Apa output program jika input berbeda?

💡 Tips Analisis

- Baca kode dari atas ke bawah
- Ikuti alur kondisi if/else
- Coba dengan nilai input yang berbeda



while — Iteration Statement

Pernyataan iterasi `while` memungkinkan program **mengulangi suatu tindakan** selama kondisi tertentu tetap bernilai benar. Begitu kondisi menjadi salah, perulangan berhenti dan eksekusi berlanjut ke pernyataan berikutnya.

Sintaks while

```
while (kondisi) {  
    // pernyataan yang diulang  
    // selama kondisi = true  
}
```

Contoh Sederhana

```
int hitungan = 1;  
while (hitungan <= 5) {  
    System.out.println(hitungan);  
    hitungan++;  
}
```

Alur Kerja while

Java pertama-tama mengevaluasi kondisi. Jika **true**, blok dieksekusi, lalu kondisi dievaluasi kembali. Proses ini berulang sampai kondisi menjadi **false**.

⚠️ Pastikan kondisi akhirnya menjadi false, atau program akan terjebak dalam **infinite loop** (perulangan tak terbatas)!

Counter-Controlled Iteration

Pada **counter-controlled iteration**, jumlah iterasi sudah diketahui sebelumnya. Sebuah variabel counter digunakan untuk menghitung berapa kali loop sudah berjalan.

Kasus: Rata-rata Nilai Siswa

Dua pendekatan untuk menghitung rata-rata nilai:

Pendekatan 1: Integer Division

Menggunakan pembagian bilangan bulat. Hasil mungkin kehilangan desimal jika tidak di-cast ke double.

Pendekatan 2: Casting

Mengonversi nilai ke double sebelum pembagian agar hasil rata-rata memiliki nilai desimal yang akurat.

Contoh Kode

```
int total = 0;
int counter = 1;
while (counter <= 10) {
    System.out.print("Masukkan nilai: ");
    int nilai = scanner.nextInt();
    total += nilai;
    counter++;
}
double rata = (double) total / 10;
System.out.println("Rata-rata: " + rata);
```

Sentinel-Controlled Iteration

Berbeda dengan counter-controlled, pada **sentinel-controlled iteration** jumlah data tidak diketahui sebelumnya. Program terus menerima input hingga pengguna memasukkan **nilai sentinel** — nilai khusus yang menandakan akhir input.

Contoh Kode Sentinel

```
int total = 0, counter = 0;
System.out.print("Masukkan nilai (-1 berhenti): ");
int nilai = scanner.nextInt();

while (nilai != -1) {
    total += nilai;
    counter++;
    System.out.print("Masukkan nilai: ");
    nilai = scanner.nextInt();
}

if (counter != 0)
    System.out.printf("Rata-rata: %.2f%n",
        (double) total / counter);
```

Konsep Nilai Sentinel

Nilai sentinel adalah nilai yang **tidak mungkin menjadi data valid**. Dalam kasus nilai ujian (0–100), nilai -1 adalah pilihan sempurna sebagai sentinel.



Sentinel-controlled iteration sangat berguna saat jumlah data bervariasi dan tidak diketahui di awal program.

Study Case: Nested Control Statements

Sebuah perguruan tinggi ingin menganalisis hasil ujian lisensi **10 siswa** broker real estat. Program harus membaca hasil setiap siswa (1 = Lulus, 2 = Gagal) dan merangkum hasilnya.

1

Input Hasil Tes

Masukkan setiap hasil tes (1 atau 2) dengan prompt "Masukkan hasil" untuk setiap input.

2

Hitung Tiap Jenis

Program menghitung jumlah siswa yang lulus (nilai 1) dan jumlah yang gagal (nilai 2) secara terpisah.

3

Tampilkan Ringkasan

Tampilkan total siswa lulus dan gagal sebagai ringkasan akhir hasil ujian.

4

Bonus Instruktur

Jika lebih dari 8 siswa lulus, cetak pesan "**Bonus untuk instruktur!**" sebagai apresiasi.

Implementasi: Nested Control Statements

Berikut adalah implementasi Java dari studi kasus ujian real estat. Perhatikan bagaimana `while` loop bersarang dengan `if...else` untuk menghasilkan logika yang lengkap.

Kode Program

```
int lulus = 0, gagal = 0;
int siswa = 1;
while (siswa <= 10) {
    System.out.print("Masukkan hasil: ");
    int hasil = scanner.nextInt();
    if (hasil == 1)
        lulus++;
    else
        gagal++;
    siswa++;
}
System.out.println("Lulus: " + lulus);
System.out.println("Gagal: " + gagal);
if (lulus > 8)
    System.out.println(
        "Bonus untuk instruktur!");
```

Analisis Struktur

- **while** — mengulang 10 kali untuk 10 siswa
- **if...else** — menentukan lulus atau gagal per siswa
- **if terakhir** — menentukan bonus instruktur

✓ Nested control statements memungkinkan logika kompleks dibangun dari komponen sederhana yang saling bersarang.

Compound Assignment Operators

Operator penugasan gabungan (*compound assignment operators*) memungkinkan kita **menyingkat ekspresi penugasan** yang melibatkan operasi aritmatika pada variabel yang sama. Ini membuat kode lebih ringkas dan mudah dibaca.

+= (Tambah)

```
c = c + 3;  
// sama dengan:  
c += 3;
```

-= (Kurang)

```
c = c - 3;  
// sama dengan:  
c -= 3;
```

***= (Kali)**

```
c = c * 3;  
// sama dengan:  
c *= 3;
```

/= (Bagi)

```
c = c / 3;  
// sama dengan:  
c /= 3;
```

%= (Modulo)

```
c = c % 3;  
// sama dengan:  
c %= 3;
```

Contoh nyata: `total += nilai;` adalah cara ringkas untuk menulis `total = total + nilai;` yang sering digunakan dalam loop akumulasi.

Increment dan Decrement Operators

Java menyediakan dua operator unary khusus untuk **menambah atau mengurangi nilai variabel sebesar 1**. Operator ini sangat sering digunakan dalam perulangan dan penghitungan sederhana.

Dua Jenis Operator

++ (Increment)

Menambahkan 1 ke nilai variabel.
Pengganti dari `c = c + 1` atau `c += 1`.

-- (Decrement)

Mengurangi 1 dari nilai variabel.
Pengganti dari `c = c - 1` atau `c -= 1`.

Prefix vs Postfix

Notasi	Nama	Kapan Diubah
<code>++c</code>	Prefix increment	Sebelum digunakan
<code>c++</code>	Postfix increment	Setelah digunakan
<code>--c</code>	Prefix decrement	Sebelum digunakan
<code>c--</code>	Postfix decrement	Setelah digunakan



Perbedaan prefix dan postfix sangat penting saat operator digunakan dalam ekspresi yang lebih kompleks!

Prefix vs Postfix: Perbedaan Nyata

Untuk memahami perbedaan prefix dan postfix, perhatikan contoh berikut dengan seksama. Urutan operasi akan menghasilkan output yang berbeda!

Contoh Prefix (++c)

```
int c = 5;  
System.out.println(++c);  
// Output: 6  
// c diubah DULU, baru dicetak  
  
System.out.println(c);  
// Output: 6
```

Contoh Postfix (c++)

```
int c = 5;  
System.out.println(c++);  
// Output: 5  
// c dicetak DULU, baru diubah  
  
System.out.println(c);  
// Output: 6
```



Code Analysis: Apa yang Dilakukan Kode Berikut?

Sebagai penutup pertemuan ini, mari kita lakukan analisis kode secara mandiri. Ini adalah latihan penting untuk mengasah kemampuan membaca dan memahami alur logika program Java.

Materi yang Sudah Dipelajari

- Algoritma dan Pseudocode
- Sequence, Selection, Iteration Statements
- if, if...else, Nested if...else
- while loop & Counter/Sentinel Iteration
- Compound & Increment/Decrement Operators

Tantangan Analisis

Perhatikan kode yang diberikan oleh instruktur. Identifikasi:

- Jenis control statement yang digunakan
- Alur eksekusi program step-by-step
- Output yang dihasilkan untuk input tertentu
- Potensi bug atau masalah logika

-  Kemampuan membaca dan menganalisis kode orang lain adalah skill krusial seorang programmer profesional. Teruslah berlatih!

